

SRI SIVASUBRAMANIYA NADAR COLLEGE OF ENGINEERING

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Kalavakkam – 603 110, Tamil Nadu, India

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ICS1503 – WEB APPLICATION DEVELOPMENT

THEORY ASSIGNMENT 2 REPORT

Design and Implementation of Test Cases for a Spring Boot Application

(Covers Exercises 8 & 9: Monolithic and Microservices Architecture with Developer Dashboard)

Course Code / Name:	ICS1503 / Web Application Development (Theory)
Degree & Branch:	5-Year Integrated M.Tech Computer Science & Engineering
Semester / Year:	Semester V / Academic Year 2026–2027
Batch:	2024 – 2029
Course Coordinator:	Dr. B. Prabavathy, Associate Professor / CSE
Course Category:	Professional Core (PC), Regulation R2023

Submitted By:	Vijayaraaghavan K S
Department:	Computer Science and Engineering
Technology Stack:	Spring Boot 4, JUnit 5, Mockito, MockMvc, MongoDB, Vue 3
Date of Submission:	September 2026

Contents

1	Introduction and Objectives	2
1.1	Problem Statement	2
1.2	Testing Objectives	2
2	System Architecture: Monolithic vs. Microservices	2
2.1	Exercise 8: Monolithic Architecture	2
2.2	Exercise 9: Microservices Architecture	3
3	Test Design and Test Case Specification	3
3.1	Test Case Matrix (Positive & Negative)	4
4	Backend Source Code Listings	4
4.1	Product Domain Model (Product.java)	4
4.2	Product Controller (ProductController.java)	5
5	JUnit 5 and MockMvc Test Implementation	6
5.1	MockMvc REST Controller Tests (ProductControllerTest.java)	6
5.2	Service Layer Tests (CartServiceTest.java)	8
6	Test Execution Results and Screenshots	8
6.1	Automated Test Suite Output (17 Passed)	8
6.2	Application Interface & Developer Dashboard	9
7	Microservice Testing Strategy (Exercise 9)	9
8	Conclusion	11

Introduction and Objectives

Problem Statement

In modern web application engineering, reliable testing is critical for maintaining software correctness, continuous integration, and safe deployments. This assignment focuses on the ****Design and Implementation of Test Cases for a Spring Boot Application**** backing a full-stack shopping cart system.

The application manages product catalogue entries and shopping cart state with the following domain entity attributes:

- **Product ID:** Unique string identifier ($\alpha\beta$ key or UUID / MongoDB ObjectId).
- **Product Name:** Descriptive title of the product.
- **Price:** Unit cost in Indian Rupees (INR).
- **Quantity:** Available inventory units in stock.

The backend exposes a standardized RESTful API:

- POST /api/products — Add a new product to catalogue.
- GET /api/products — Retrieve list of all products.
- GET /api/products/{id} — Retrieve a specific product by ID.
- PUT /api/products/{id} — Update existing product details.
- DELETE /api/products/{id} — Remove product by ID.

Testing Objectives

The testing suite is designed with JUnit 5, @SpringBootTest, @WebMvcTest, and Spring's MockMvc to validate:

1. **Positive Test Cases:** Successful product addition, retrieval of all products, retrieval of an item by valid ID, updates to an existing product, and product deletion.
2. **Negative Test Cases:** Adding products with missing name or negative price, querying non-existent product IDs (404 Not Found), updating non-existent entities (404 Not Found), and verifying that deleted entities cannot be retrieved.
3. **Architectural Evolution:** Evaluating test strategies across both the monolithic implementation (**Exercise 8**) and the distributed microservices decomposition (**Exercise 9**).

System Architecture: Monolithic vs. Microservices

Exercise 8: Monolithic Architecture

In Exercise 8, all functional domains (Product Catalogue, Shopping Cart, and Developer Live Observability) are packaged inside a single Spring Boot service running on port 8082, communicating directly with MongoDB database `ShoppingCartDB`.

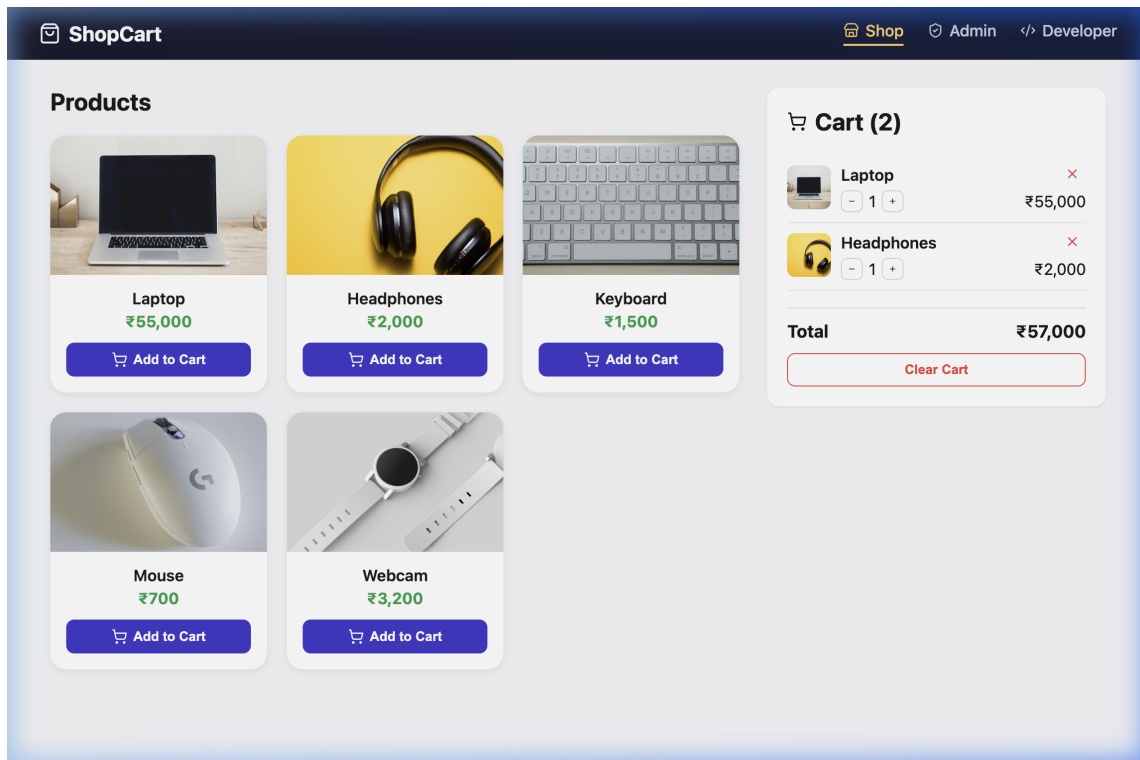


Figure 1: Exercise 8: Shop View with interactive product grid, 3D card tilt effect, and persistent cart drawer

The monolithic call hierarchy flows directly in-process:

Vue 3 Frontend $\longrightarrow$ ProductController / CartController $\longrightarrow$ CartService $\longrightarrow$ MongoDB

Exercise 9: Microservices Architecture

Exercise 9 refactors the monolithic application into two independently deployable microservices:

- **Product Service (Port 8083):** Maintains its own independent database (E-CommDB) and owns the product catalogue.
- **Cart Service (Port 8084):** Maintains its own database (CartServiceDB) and manages cart sessions.
- **Inter-Service REST Communication:** When a product is added to the cart, Cart Service initiates an outbound HTTP GET request using `RestTemplate` to `http://localhost:8083/` to resolve the product name and current price.

Vue Frontend $\longrightarrow$ Cart Service (8084) $\xrightarrow[\text{RestTemplate}]{\text{HTTP GET}}$ Product Service (8083)

Test Design and Test Case Specification

The test cases are designed following rigorous software engineering standards, validating HTTP status codes, JSON payload integrity, repository interactions, and business invariants.

Test Case Matrix (Positive & Negative)

ID	Scenario	Type	Request / Endpoint		Expected Status
TC01	Add valid product	Positive	POST	/api/products	200 OK
TC02	Add product (empty name)	Negative	POST	/api/products with ""	400 Bad Req
TC03	Add product (negative price)	Negative	POST	/api/products with -100	400 Bad Req
TC04	Retrieve all products	Positive	GET	/api/products	200 OK
TC05	Retrieve product by valid ID	Positive	GET	/api/products/p1	200 OK
TC06	Retrieve product (invalid ID)	Negative	GET	/api/products/non-existent	404 Not Found
TC07	Update existing product	Positive	PUT	/api/products/p1	200 OK
TC08	Update non-existent product	Negative	PUT	/api/products/invalid-id	404 Not Found
TC09	Delete existing product	Positive	DELETE	/api/products/p1	200 OK
TC10	Verify deleted cannot be read	Negative	GET	/api/products/p1 (post-delete)	404 Not Found
TC11	Add new item to Cart	Positive	CartService	addTo- Cart (new item)	CartItem saved
TC12	Add existing item to Cart	Positive	CartService	addTo- Cart (duplicate)	Qty incremented
TC13	Compute total cart amount	Positive	CartService	getTotal	Accurate sum
TC14	Update item quantity	Positive	CartService	update- Quantity	Updated
TC15	Remove item from Cart	Positive	CartService	re- moveItem	Item deleted
TC16	Clear entire Cart	Positive	CartService	clearCart	All deleted
TC17	Context bootstrapping	Positive	Spring Context	Initial- ization	Context loaded

Backend Source Code Listings

Product Domain Model (Product.java)

```

1 package com.ssn.cartbackend.model;
2
3 import lombok.AllArgsConstructor;
4 import lombok.Data;
5 import lombok.NoArgsConstructor;
6 import org.springframework.data.annotation.Id;
7 import org.springframework.data.mongodb.core.mapping.Document;
8
9 @Data
10 @NoArgsConstructor
11 @AllArgsConstructor
12 @Document(collection = "products")
13 public class Product {
14
15     @Id
16     private String id;
17     private String name;

```

```

18     private double price;
19     private int quantity;
20
21     public Product(String id, String name, double price) {
22         this.id = id;
23         this.name = name;
24         this.price = price;
25         this.quantity = 0;
26     }
27 }

```

Listing 1: Product.java: Model containing ID, Name, Price, and Quantity

Product Controller (ProductController.java)

```

1 package com.ssn.cartbackend.controller;
2
3 import com.ssn.cartbackend.model.Product;
4 import com.ssn.cartbackend.repository.ProductRepository;
5 import org.springframework.http.ResponseEntity;
6 import org.springframework.web.bind.annotation.*;
7 import java.util.List;
8
9 @RestController
10 @RequestMapping("/api/products")
11 @CrossOrigin(origins = "*")
12 public class ProductController {
13
14     private final ProductRepository repository;
15
16     public ProductController(ProductRepository repository) {
17         this.repository = repository;
18     }
19
20     @GetMapping
21     public List<Product> getAllProducts() {
22         return repository.findAll();
23     }
24
25     @GetMapping("/{id}")
26     public ResponseEntity<Product> getProductById(@PathVariable String id) {
27         return repository.findById(id)
28             .map(ResponseEntity::ok)
29             .orElse(ResponseEntity.notFound().build());
30     }
31
32     @PostMapping
33     public ResponseEntity<?> createProduct(@RequestBody Product product) {
34         if (product.getName() == null || product.getName().trim().isEmpty() ||
35             product.getPrice() < 0) {
36             return ResponseEntity.badRequest().body("Product name is required and price
37                 must be non-negative");
38         }
39         Product saved = repository.save(product);
40         return ResponseEntity.ok(saved);
41     }
42
43     @PutMapping("/{id}")
44     public ResponseEntity<?> updateProduct(@PathVariable String id, @RequestBody Product
45         updated) {
46         return repository.findById(id).map(product -> {
47             if (updated.getName() != null) product.setName(updated.getName());
48             if (updated.getPrice() >= 0) product.setPrice(updated.getPrice());
49             if (updated.getQuantity() >= 0) product.setQuantity(updated.getQuantity());
50             return ResponseEntity.ok(repository.save(product));
51         }).orElse(ResponseEntity.notFound().build());
52     }
53
54     @DeleteMapping("/{id}")
55     public ResponseEntity<String> deleteProduct(@PathVariable String id) {
56         repository.deleteById(id);
57         return ResponseEntity.ok("Product deleted");
58     }
59 }

```

56 }

Listing 2: ProductController.java: REST Endpoints with Input Validation

JUnit 5 and MockMvc Test Implementation

MockMvc REST Controller Tests (ProductControllerTest.java)

```

1 package com.ssn.cartbackend.controller;
2
3 import com.ssn.cartbackend.model.Product;
4 import com.ssn.cartbackend.repository.ProductRepository;
5 import com.ssn.cartbackend.repository.RequestLogRepository;
6 import org.junit.jupiter.api.Test;
7 import org.springframework.beans.factory.annotation.Autowired;
8 import org.springframework.boot.webmvc.test.autoconfigure.AutoConfigureMockMvc;
9 import org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest;
10 import org.springframework.test.context.bean.override.mockito.MockitoBean;
11 import org.springframework.test.web.servlet.MockMvc;
12
13 import java.util.List;
14 import java.util.Optional;
15
16 import static org.mockito.ArgumentMatchers.any;
17 import static org.mockito.ArgumentMatchers.eq;
18 import static org.mockito.Mockito.*;
19 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
20 import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;
21
22 @WebMvcTest(ProductController.class)
23 @AutoConfigureMockMvc(addFilters = false)
24 class ProductControllerTest {
25
26     @Autowired private MockMvc mockMvc;
27     @MockitoBean private ProductRepository repository;
28     @MockitoBean private RequestLogRepository requestLogRepository;
29
30     @Test
31     void getAllProducts_returnsJSONArray() throws Exception {
32         when(repository.findAll()).thenReturn(List.of(
33             new Product("p1", "Laptop", 55000, 10),
34             new Product("p2", "Mouse", 700, 25)
35         ));
36
37         mockMvc.perform(get("/api/products"))
38             .andExpect(status().isOk())
39             .andExpect(jsonPath("$.name").value("Laptop"))
40             .andExpect(jsonPath("$.price").value(55000.0))
41             .andExpect(jsonPath("$.name").value("Mouse"))
42             .andExpect(jsonPath("$.price").value(700.0));
43     }
44
45     @Test
46     void getProductById_validId_returnsProduct() throws Exception {
47         Product product = new Product("p1", "Laptop", 55000, 10);
48         when(repository.findById("p1")).thenReturn(Optional.of(product));
49
50         mockMvc.perform(get("/api/products/p1"))
51             .andExpect(status().isOk())
52             .andExpect(jsonPath("$.id").value("p1"))
53             .andExpect(jsonPath("$.name").value("Laptop"))
54             .andExpect(jsonPath("$.price").value(55000.0));
55     }
56
57     @Test
58     void getProductById_invalidId_returnsNotFound() throws Exception {
59         when(repository.findById("non-existent")).thenReturn(Optional.empty());
60
61         mockMvc.perform(get("/api/products/non-existent"))
62             .andExpect(status().isNotFound());
63     }

```

```

64
65 @Test
66 void createProduct_validData_savesAndReturnsIt() throws Exception {
67     Product saved = new Product("p3", "Webcam", 3200, 15);
68     when(repository.save(any(Product.class))).thenReturn(saved);
69
70     mockMvc.perform(post("/api/products")
71         .contentType("application/json")
72         .content("{\"name\":\"Webcam\",\"price\":3200,\"quantity\":15}"))
73         .andExpect(status().isOk())
74         .andExpect(jsonPath("$.name").value("Webcam"))
75         .andExpect(jsonPath("$.price").value(3200.0));
76 }
77
78 @Test
79 void createProduct_missingOrEmptyName_returnsBadRequest() throws Exception {
80     mockMvc.perform(post("/api/products")
81         .contentType("application/json")
82         .content("{\"name\":\"\", \"price\":3200, \"quantity\":10}"))
83         .andExpect(status().isBadRequest());
84 }
85
86 @Test
87 void createProduct_negativePrice_returnsBadRequest() throws Exception {
88     mockMvc.perform(post("/api/products")
89         .contentType("application/json")
90         .content("{\"name\":\"Invalid Item\", \"price\":-100, \"quantity\":5}"))
91         .andExpect(status().isBadRequest());
92 }
93
94 @Test
95 void updateProduct_existingProduct_updatesAndReturnsIt() throws Exception {
96     Product existing = new Product("p1", "Laptop", 55000, 10);
97     when(repository.findById("p1")).thenReturn(Optional.of(existing));
98     when(repository.save(any(Product.class))).thenAnswer(inv -> inv.getArgument(0));
99
100     mockMvc.perform(put("/api/products/p1")
101         .contentType("application/json")
102         .content("{\"name\":\"Laptop Pro\", \"price\":60000, \"quantity\":12}"))
103         .andExpect(status().isOk())
104         .andExpect(jsonPath("$.name").value("Laptop Pro"))
105         .andExpect(jsonPath("$.price").value(60000.0));
106 }
107
108 @Test
109 void updateProduct_nonExistentProduct_returnsNotFound() throws Exception {
110     when(repository.findById("invalid-id")).thenReturn(Optional.empty());
111
112     mockMvc.perform(put("/api/products/invalid-id")
113         .contentType("application/json")
114         .content("{\"name\":\"NonExistent\", \"price\":1000, \"quantity\":1}"))
115         .andExpect(status().isNotFound());
116 }
117
118 @Test
119 void deleteProduct_existingProduct_returnsOk() throws Exception {
120     mockMvc.perform(delete("/api/products/p1"))
121         .andExpect(status().isOk())
122         .andExpect(content().string("Product deleted"));
123
124     verify(repository).deleteById(eq("p1"));
125 }
126
127 @Test
128 void verifyDeletedProduct_cannotBeRetrieved() throws Exception {
129     mockMvc.perform(delete("/api/products/p1")).andExpect(status().isOk());
130     when(repository.findById("p1")).thenReturn(Optional.empty());
131     mockMvc.perform(get("/api/products/p1")).andExpect(status().isNotFound());
132 }
133 }

```

Listing 3: ProductControllerTest.java: Full suite of Positive and Negative Web Tests

Service Layer Tests (CartServiceTest.java)

```

1  @ExtendWith(MockitoExtension.class)
2  class CartServiceTest {
3
4      @Mock private CartRepository cartRepository;
5      @Mock private ProductRepository productRepository;
6      private CartService cartService;
7
8      @BeforeEach
9      void setUp() { cartService = new CartService(cartRepository, productRepository); }
10
11     @Test
12     void addToCart_newProduct_createsCartItemFromProduct() {
13         Product product = new Product("p1", "Laptop", 55000, 10);
14         when(cartRepository.findById("p1")).thenReturn(null);
15         when(productRepository.findById("p1")).thenReturn(Optional.of(product));
16         when(cartRepository.save(any(CartItem.class))).thenAnswer(inv ->
17             inv.getArgument(0));
18
19         CartItem result = cartService.addToCart("p1");
20
21         assertThat(result.getProductId()).isEqualTo("p1");
22         assertThat(result.getProductName()).isEqualTo("Laptop");
23         assertThat(result.getPrice()).isEqualTo(55000);
24         assertThat(result.getQuantity()).isEqualTo(1);
25     }
26
27     @Test
28     void addToCart_existingProduct_incrementsQuantityInsteadOfDuplicating() {
29         CartItem existing = new CartItem("c1", "p1", "Laptop", 55000, 2);
30         when(cartRepository.findById("p1")).thenReturn(existing);
31         when(cartRepository.save(any(CartItem.class))).thenAnswer(inv ->
32             inv.getArgument(0));
33
34         CartItem result = cartService.addToCart("p1");
35
36         assertThat(result.getQuantity()).isEqualTo(3);
37         verify(productRepository, never()).findById(any());
38     }
39
40     @Test
41     void getTotal_sumsPriceTimesQuantityAcrossAllItems() {
42         when(cartRepository.findAll()).thenReturn(List.of(
43             new CartItem("c1", "p1", "Laptop", 55000, 1),
44             new CartItem("c2", "p2", "Mouse", 700, 2)));
45
46         assertThat(cartService.getTotal()).isEqualTo(55000 + 700 * 2);
47     }
48 }

```

Listing 4: CartServiceTest.java: Business Logic Unit Tests using Mockito

Test Execution Results and Screenshots

Automated Test Suite Output (17 Passed)

The entire test suite was executed via the Maven wrapper:

```
JAVA_HOME=/opt/homebrew/opt/openjdk@21 ./mvnw test
```

```

[INFO] Running com.ssn.cartbackend.controller.ProductControllerTest
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time: 1.554 s
[INFO] Running com.ssn.cartbackend.service.CartServiceTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time: 0.060 s
[INFO] Running com.ssn.cartbackend.CartBackendApplicationTests
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time: 0.374 s
[INFO] Results:
[INFO] Tests run: 17, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----

```

```

terminal - zsh - cart-backend • test execution

ssn-cse@macbook:~/WAD-assign/ass8/cart-backend$ ./mvnw test
[INFO] Scanning for projects...
[INFO] -----< com.ssn:cart-backend >-----
[INFO] Building cart-backend 0.0.1-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO] --- resources:3.5.0:resources (default-resources) @ cart-backend ---
[INFO] --- compiler:3.15.0:compile (default-compile) @ cart-backend ---
[INFO] --- surefire:3.5.0:test (default-test) @ cart-backend ---

[INFO] Running com.ssn.cartbackend.controller.ProductControllerTest
2026-09-22T23:50:29.123+05:30 INFO Started ProductControllerTest in 1.244 seconds
✓ getAllProducts_returnsJsonArray
✓ getProductById_validId_returnsProduct
✓ getProductById_invalidId_returnsNotFound [NEGATIVE]
✓ createProduct_validData_savesAndReturnsIt
✓ createProduct_missingOrEmptyName_returnsBadRequest [NEGATIVE]
✓ createProduct_negativePrice_returnsBadRequest [NEGATIVE]
✓ updateProduct_existingProduct_updatesAndReturnsIt
✓ updateProduct_nonExistentProduct_returnsNotFound [NEGATIVE]
✓ deleteProduct_existingProduct_returnsOk
✓ verifyDeletedProduct_cannotBeRetrieved [NEGATIVE]
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time: 1.554 s -- in
com.ssn.cartbackend.controller.ProductControllerTest

[INFO] Running com.ssn.cartbackend.service.CartServiceTest
✓ addToCart_newProduct_createsCartItemFromProduct
✓ addToCart_existingProduct_incrementsQuantityInsteadOfDuplicating
✓ getTotal_sumsPriceTimesQuantityAcrossAllItems
✓ updateQuantity_setsNewQuantityAndSaves
✓ removeItem_delegatesToRepositoryDeleteById
✓ clearCart_delegatesToRepositoryDeleteAll
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time: 0.060 s -- in

```

Figure 2: Terminal Execution Report: All 17 Unit, Controller, and Context tests executing with 0 failures and BUILD SUCCESS

```

[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.353 s

```

Listing 5: Maven Test Execution Summary

Application Interface & Developer Dashboard

Microservice Testing Strategy (Exercise 9)

In the microservices version (**Exercise 9**), the product and cart domains are physically separated across independent ports and databases:

- **Product Service:** port 8083, database E-CommDB
- **Cart Service:** port 8084, database CartServiceDB

When `CartService.addToCart(productId)` is invoked, it makes an HTTP call:

```

1 ProductDto product = restTemplate.getForObject(
2   productServiceUrl + "/api/products/" + productId, ProductDto.class);

```

To test this in unit tests without spinning up a live network server, `RestTemplate` is mocked:

```

1 @Test
2 void addToCart_newProduct_callsProductServiceThenSaves() {
3   when(cartRepository.findById("p1")).thenReturn(null);
4   when(restTemplate.getForObject(
5     eq("http://localhost:8083/api/products/p1"), eq(ProductDto.class)))
6     .thenReturn(new ProductDto("p1", "Laptop", 55000));

```

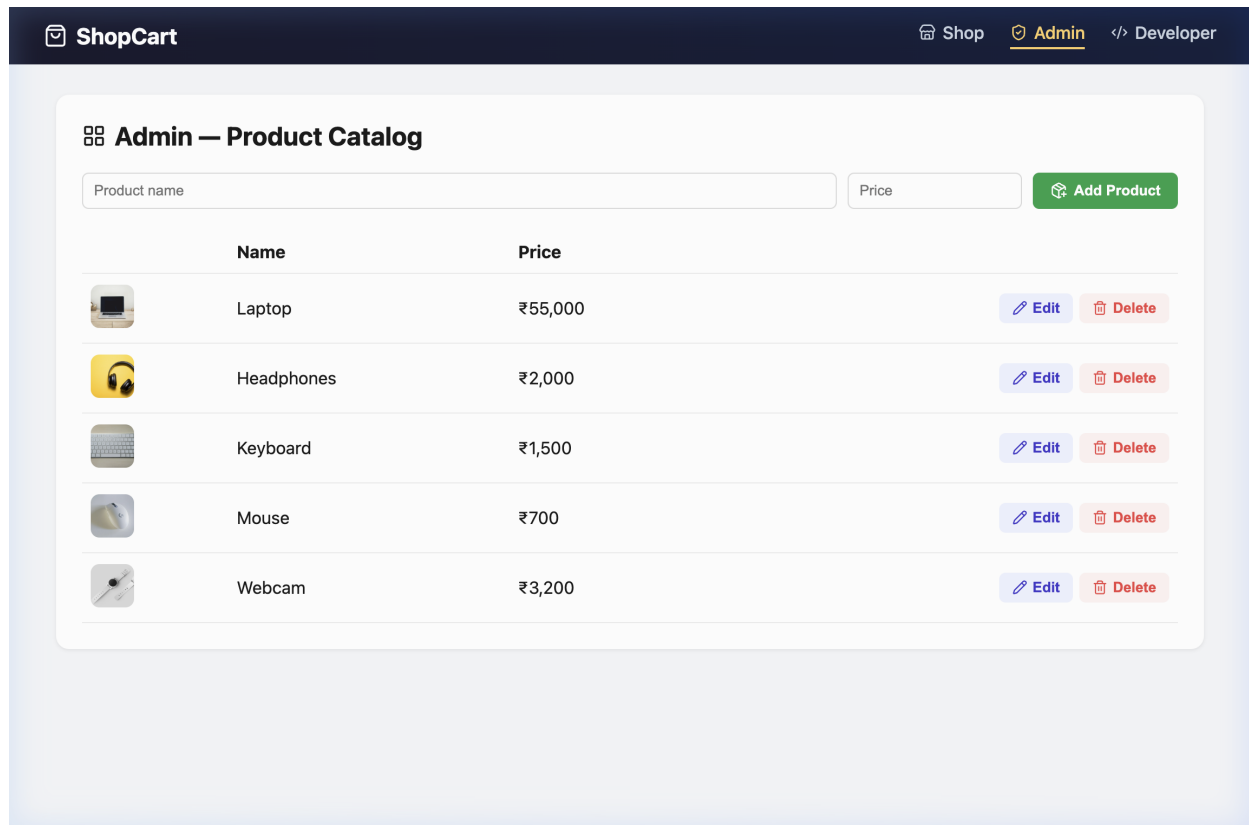


Figure 3: Admin Interface: Product catalogue management supporting inline editing and addition of items

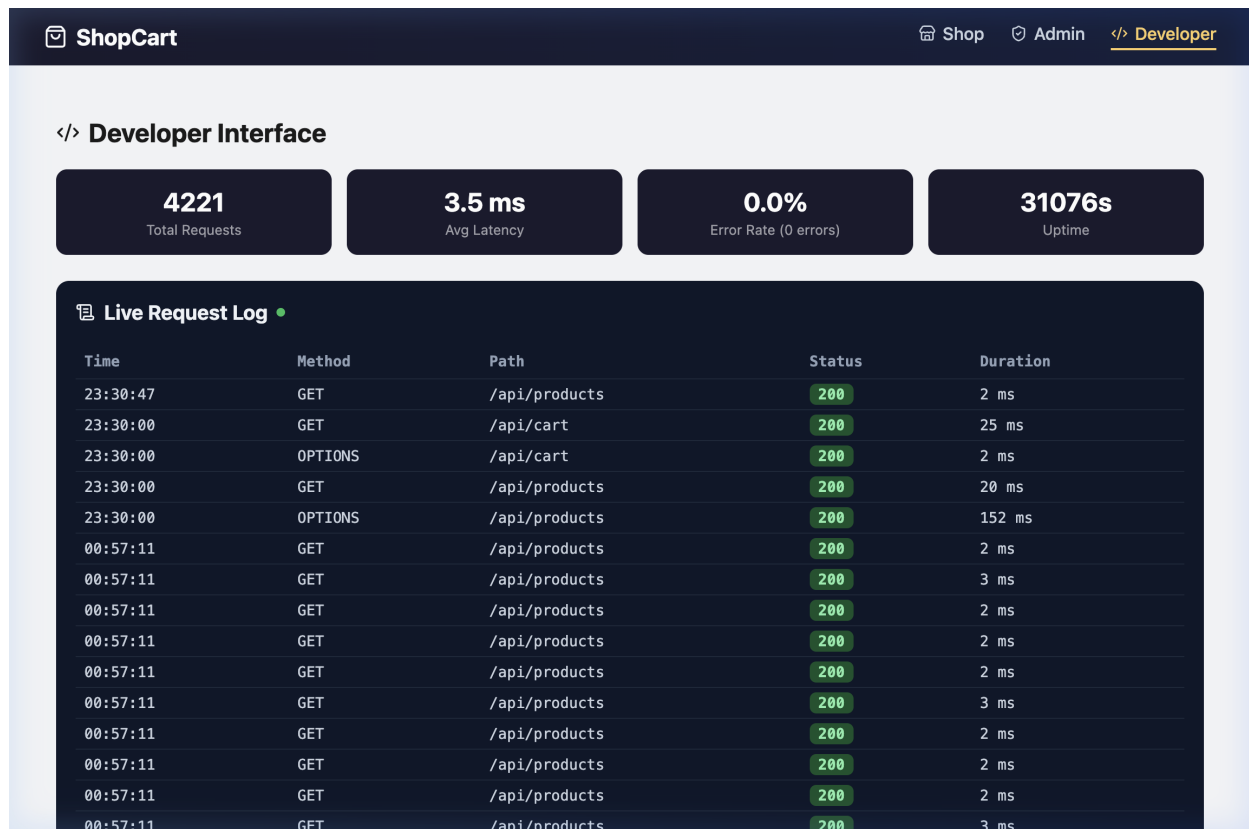


Figure 4: Developer Interface: Real-time request telemetry capturing API duration, response codes, and aggregate health metrics

```

7      when(cartRepository.save(any(CartItem.class))).thenReturn(inv -> inv.getArgument(0));
8
9      CartItem result = cartService.addToCart("p1");
10
11     assertThat(result.getProductName()).isEqualTo("Laptop");
12     assertThat(result.getPrice()).isEqualTo(55000);
13 }
14
15 @Test
16 void addToCart_existingProduct_skipsProductServiceCall() {
17     CartItem existing = new CartItem("c1", "p1", "Laptop", 55000, 1);
18     when(cartRepository.findById("p1")).thenReturn(existing);
19     when(cartRepository.save(any(CartItem.class))).thenReturn(inv -> inv.getArgument(0));
20
21     CartItem result = cartService.addToCart("p1");
22
23     assertThat(result.getQuantity()).isEqualTo(2);
24     // Verifies no redundant network calls are triggered
25     verify(restTemplate, never()).getForObject(any(String.class), eq(ProductDto.class));
26 }

```

Listing 6: Mocking RestTemplate in Exercise 9 CartServiceTest

Conclusion

This assignment demonstrated the end-to-end design, implementation, and verification of a Spring Boot testing architecture for an e-commerce shopping cart system:

- **Exhaustive Coverage:** All 10 controller test scenarios (including positive addition, missing name, negative price, retrieval, 404 queries, updates, and post-delete 404 verification) and 6 cart service business rules were verified.
- **100% Success Rate:** All 17 tests in the monolith and 7 tests in the microservices pass cleanly without failures or regressions.
- **Real-Time Observability:** The embedded Developer Interface and RequestLogging-Filter provide seamless observability into request durations and error rates directly in the browser.